

🤔 323 - Aide-mémoire Programmation fonctionnelle

💡 Tip

Sujet : Programmation fonctionnelle

Fait par : Paul Friedli

Version : 26.06.2025

Référence : <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference>

Tester du code JS : <https://runjs.app/play>

Convertir en PDF : <https://marketplace.visualstudio.com/items?itemName=manuth.markdown-converter>

Table des matières - par domaine

- **Date et Heure**
 - `Obtenir la date et/ou heure actuelle`
- **Math**
 - `Math.PI` - la constante π
 - `Math.abs()` - la |valeur absolue| d'un nombre
 - `Math.pow()` - élever à une puissance
 - `Math.min()` - plus petite valeur
 - `Math.max()` - plus grande valeur
 - `Math.ceil()` - arrondir à la prochaine valeur entière la plus proche
 - `Math.floor()` - arrondir à la précédente valeur entière la plus proche
 - `Math.round()` - arrondir à la valeur entière la plus proche
 - `Math.trunc()` - supprime la virgule et retourne la partie entière d'un nombre
 - `Math.sqrt()` - la racine carrée d'un nombre
 - `Math.random()` - générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)
- **JSON**
 - `JSON.stringify()` - transformer un objet Javascript en JSON
 - `JSON.parse()` - transformer du JSON en objet Javascript
- **Chaînes de caractères**
 - `split()` - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau
 - `trim()`, `trimStart()` et `trimEnd()` - épuration des espaces en trop dans une chaîne (trimming)
 - `padStart()` et `padEnd()` - aligner le contenu dans une chaîne de caractères
- **Console**
 - `console.log()` - Afficher un message sur la console
 - `console.info()`, `warn()` et `error()` - Afficher un message sur la console (filtrables)
 - `console.table()` - Afficher tout un tableau ou un objet sur la console
 - `console.time()`, `timeLog()` et `timeEnd()` - Chronométrer une durée d'exécution
- **Tableaux**
 - `forEach` - parcourir les éléments d'un tableau
 - `entries()` - parcourir les couples index/valeurs d'un tableau
 - `in` - parcourir les clés d'un tableau
 - `of` - parcourir les valeurs d'un tableau
 - `find()` - premier élément qui satisfait une condition
 - `findIndex()` - premier index qui satisfait une condition
 - `indexOf()` et `lastIndexOf()` - premier/dernier élément qui correspond
 - `push()`, `pop()`, `shift()` et `unshift()` - ajouter/supprime au début/fin dans un tableau
 - `slice()` - ne conserver que certaines lignes d'un tableau
 - `splice()` - supprimer/insérer/remplacer des valeurs dans un tableau
 - `concat()` - joindre deux tableaux
 - `join()` - joindre des chaînes de caractères
 - `keys()` et `values()` - les clés/valeurs d'un objet

- `includes()` - vérifier si une valeur est présente dans un tableau
- `every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau
- `fill()` - remplir un tableau avec des valeurs
- `flat()` - aplatir un tableau
- `sort()` - pour trier un tableau
- `map()` - tableau avec les résultats d'une fonction
- `filter()` - tableau avec les éléments passant un test
- `groupBy()` - regroupe les éléments d'un tableau selon un règle
- `flatMap()` - chaînage de `map()` et `flat()`
- `reduce()` et `reduceRight()` - réduire un tableau à une seule valeur
- `reverse()` - inverser l'ordre du tableau
- Techniques
 - ```(backticks) - pour des expressions intelligentes
 - `new Set()` - pour supprimer les doublons
- Opérateurs javascript super-coool 😎
 - opérateur `?:`
 - opérateur `??`
 - opérateur `??=`
 - opérateur de décomposition 'spread' `...`
 - Déstructuration
- Fonctions
 - Déclaration de fonction
 - Fonctions immédiatement invoquées (IIFE)

Table des matières - avec logos

Domaine		Comment	Quoi
Date			
			Obtenir la date actuelle
			Obtenir l'heure actuelle
Math			
	Π	<code>Math.PI</code>	la constante π
	$\neq$	<code>Math.abs()</code>	la valeur absolue d'un nombre
	$\wedge$	<code>Math.pow()</code>	élever à une puissance
	$\leftarrow$	<code>Math.min()</code>	plus petite valeur
	$\rightarrow$	<code>Math.max()</code>	plus grande valeur
	$\sim \uparrow$	<code>Math.ceil()</code>	arrondir à la prochaine valeur entière la plus proche
	$\sim \downarrow$	<code>Math.floor()</code>	arrondir à la précédente valeur entière la plus proche
	$\sim$	<code>Math.round()</code>	arrondir à la valeur entière la plus proche
	$\leq$	<code>Math.trunc()</code>	supprime la virgule et retourne la partie entière d'un nombre
	$\sqrt{\quad}$	<code>Math.sqrt()</code>	la racine carrée d'un nombre
		<code>Math.random()</code>	générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)

Domaine		Comment	Quoi
JSON			
		<code>JSON.stringify()</code>	transformer un objet Javascript en JSON
		<code>JSON.parse()</code>	transformer du JSON en objet Javascript
String			
		<code>split()</code>	un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau
		<code>trim()</code> , <code>trimStart()</code> et <code>trimEnd()</code>	épuration des espaces en trop dans une chaîne (trimming)
		<code>padStart()</code> et <code>padEnd()</code>	aligner le contenu dans une chaîne de caractères
Console			
		<code>console.log()</code>	Afficher un message sur la console
		<code>console.info()</code> , <code>warn()</code> et <code>error()</code>	Afficher un message sur la console (filtrables)
		<code>console.table()</code>	Afficher tout un tableau ou un objet sur la console
		<code>console.time()</code> , <code>timeLog()</code> et <code>timeEnd()</code>	Chronométrer une durée d'exécution
Tableaux			
		<code>forEach</code>	parcourir les éléments d'un tableau
		<code>entries()</code>	parcourir les couples index/valeurs d'un tableau
		<code>in</code>	parcourir les clés d'un tableau
		<code>of</code>	parcourir les valeurs d'un tableau
		<code>find()</code>	premier élément qui satisfait une condition
		<code>findIndex()</code>	premier index qui satisfait une condition
		<code>indexOf()</code>	premier index qui correspond
		<code>lastIndexOf()</code>	dernier index qui correspond
		<code>push()</code> , <code>pop()</code> , <code>shift()</code> et <code>unshift()</code>	ajouter/supprime au début/fin dans un tableau
		<code>slice()</code>	ne conserver que certaines lignes d'un tableau
		<code>splice()</code>	supprimer/insérer/remplacer des valeurs dans un tableau
		<code>concat()</code>	joindre deux tableaux
		<code>join()</code>	joindre des chaînes de caractères
		<code>keys()</code> et <code>values()</code>	les clés/valeurs d'un objet
		<code>includes()</code>	vérifier si une valeur est présente dans un tableau
		<code>every()</code> et <code>some()</code>	vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau
		<code>fill()</code>	remplir un tableau avec des valeurs

Domaine		Comment	Quoi
		<code>flat()</code>	aplatir un tableau
		<code>sort()</code>	pour trier un tableau
		<code>map()</code>	tableau avec les résultats d'une fonction
		<code>filter()</code>	tableau avec les éléments passant un test
		<code>groupBy()</code>	regroupe les éléments d'un tableau selon un règle
		<code>flatMap()</code>	chaînage de <code>map()</code> et <code>flat()</code>
		<code>reduce()</code> et <code>reduceRight()</code>	réduire un tableau à une seule valeur
		<code>reverse()</code>	inverser l'ordre du tableau
		<code>...</code>	l'opérateur 'spread' pour décomposer un tableau/objet en ses éléments

Date et Heure

Lien vers la documentation officielle : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Date

Obtenir la date et/ou heure actuelle

```
const maintenant = new Date(); // Obtenir L'un comme L'autre

console.log(maintenant.toLocaleDateString()); // ex: "06.06.2025"
console.log(maintenant.toLocaleTimeString()); // ex: "15:23:42"

const jour = maintenant.getDate();
const mois = maintenant.getMonth() + 1; // Attention : janvier = 0
const annee = maintenant.getFullYear();
const heure = maintenant.getHours();
const minute = maintenant.getMinutes();
const seconde = maintenant.getSeconds();
console.log(`${jour}/${mois}/${annee} - ${heure}h${minute}`);

// Au format ISO (standard international)
console.log(maintenant.toISOString()); // ex: "2025-06-06T13:23:42.123Z"
```

Math

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math

Math.PI - la constante π

```
console.log(Math.PI); // 3.141592653589793
```

Math.abs() - la |valeur absolue| d'un nombre

```
console.log( Math.abs( -4 ) );    // 4
```

Math.pow() - élever à une puissance

```
console.log(Math.pow(2, 4);      // 2x2x2x2 = 16
```

Math.min() - plus petite valeur

```
console.log(Math.min(-2, 4, 0, -3, -1)); // -3
console.log(Math.max(-2, 4, 0, -3, -1)); // 4

const array1 = [2, 3, 1];
console.log(Math.min(...array1));    // 1
```

Math.max() - plus grande valeur

```
console.log(Math.min(-2, 4, 0, -3, -1)); // -3
console.log(Math.max(-2, 4, 0, -3, -1)); // 4

const array1 = [2, 3, 1];
console.log(Math.min(...array1));    // 1
```

Math.ceil() - arrondir à la prochaine valeur entière la plus proche

Retourne le plus petit entier supérieur ou égal au nombre donné (ceil = plafond).

```
console.log(Math.ceil(0.95));      // 1
console.log(Math.ceil(4));         // 4
console.log(Math.ceil(7.004));     // 8
console.log(Math.ceil(-7.004));    // -7
```

Math.floor() - arrondir à la précédente valeur entière la plus proche

Renvoie le plus grand entier qui est inférieur ou égal à un nombre x (floor = sol).

```
console.log(Math.floor(5.95));     // 5
console.log(Math.floor(5.05));     // 5
console.log(Math.floor(5));        // 5
console.log(Math.floor(-5.05));    // -6
```

Math.round() - arrondir à la valeur entière la plus proche

Retourne la valeur d'un nombre arrondi à l'entier le plus proche.

```
console.log(Math.round(0.9)); // 1
console.log(Math.round(5.95), Math.round(5.5), Math.round(5.05)); // 6 6 5
console.log(Math.round(-5.05), Math.round(-5.5), Math.round(-5.95)); // -5 -5 -6
```

Math.trunc() - supprime la virgule et retourne la partie entière d'un nombre

Retourne la troncature entière d'un nombre en retirant sa partie décimale.

```
console.log(Math.trunc(13.37)); // 13
console.log(Math.trunc(0.123)); // 0
console.log(Math.trunc(-0.123)); // -0
```

Math.sqrt() - la racine carrée d'un nombre

```
function calcHypotenuse(a, b) { return Math.sqrt(a * a + b * b); }
console.log( calcHypotenuse(3, 4) ); // 5
```

Math.random() - générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)

Renvoie un nombre flottant pseudo-aléatoire compris dans l'intervalle [0, 1[.

```
console.log(Math.random()); // 0.00000 ... 0.99999999
```

JSON

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/JSON

JSON.stringify() - transformer un objet Javascript en JSON

JSON.stringify(obj) ou **JSON.stringify(obj, null, ' ')** permet de transformer tout objet Javascript en sa représentation JSON.

```
console.log( JSON.stringify( { x: 5, y: 6 } ) );
// Expected output: '{"x":5,"y":6}'

const obj = [{ x: 5}, true, "coucou", { y: 6 }];
console.log( JSON.stringify( obj ) );
// Expected output: ' [{"x":5},true,"coucou",{ "y":6}] '

console.log( JSON.stringify( obj , null, '  ' ) );
// Expected output:
// [
//   {
//     "x": 5
//   },
//   true,
//   "coucou",
//   {
//     "y": 6
//   }
// ]
```

JSON.parse() - transformer du JSON en objet Javascript

JSON.parse() permet de transformer une chaîne de caractère JSON en son objet Javascript correspondant.

```
const strJSON = '{"Name":"GFG","Age":22,"Department":"Computer Science and Engineering","Year":"3rd"}';
const obj = JSON.parse(strJSON);

// obj will now contain this :
const sameObj = {
  Name: "GFG",
  Age : 22,
  Department : "Computer Science and Engineering",
  Year: "3rd"
};
```

Chaînes de caractères

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String

split() - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau

split() divise une chaîne de caractères en un tableau de sous-chaînes. Une sous-chaîne est créée là où le motif fourni est présent.

```
const str = "The quick brown fox jumps over the lazy dog.";
const words = str.split(" "); // words = ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog.'];

const str2 = "Un##simple#exemple";
const words2 = str2.split("#"); // words2 = ["Un", "", "simple", "exemple"];
```

trim(), trimStart() et trimEnd() - épuraison des espaces en trop dans une chaîne (trimming)

trim(), trimStart() et trimEnd() pour supprimer les éventuels espaces, tabulations, ... au début et/ou à la fin d'une chaîne de caractères.

```
const greeting = "  Hello world!  ";
console.log(greeting.trim());           // Expected output: "Hello world!";
console.log(greeting.trimStart());      // Expected output: "Hello world!  ";
```

padStart() et padEnd() - aligner le contenu dans une chaîne de caractères

padStart() et padEnd() pour aligner le contenu de chaînes de caractères.

```
const str1 = "5";
console.log(str1.padStart(4, " "));      // Expected output: "  5"

const fullNumber = "2034399002125581";
const last4Digits = fullNumber.slice(-4);
const maskedNumber = last4Digits.padStart(fullNumber.length, "*");
console.log(maskedNumber);               // Expected output: "*****5581"

const str1 = "Breaded Mushrooms";
console.log(str1.padEnd(25, "."));      // Expected output: "Breaded Mushrooms....."

const str2 = "200";
console.log(str2.padEnd(5));             // Expected output: "200  "
```

Console

Lien vers la documentation officielle : <https://developer.mozilla.org/fr/docs/Web/API/console>

console.log() - Afficher un message sur la console

```
console.log("Coucou !");                // Coucou !
```

console.info(), warn() et error() - Afficher un message sur la console (filtrables)

```
console.info("Coucou !");
console.warn("Coucou !");
console.error("Coucou !");
```

console.table() - Afficher tout un tableau ou un objet sur la console

Affiche **un tableau** ou **un objet** de la manière la plus lisible possible sur la console ([voir documentation officielle pour des exemples visuels](#)).

```
console.table(["apples", "oranges", "bananas"]);
```

(index)	Values
0	'apples'
1	'oranges'
2	'bananas'

```
const him = new Person("Tyrone", "Jones");  
console.table(him);
```

(index)	Values
firstName	'Tyrone'
lastName	'Jones'

console.time(), timeLog() et timeEnd() - Chronométrer une durée d'exécution

```
console.time("ma-mesure");           // Démarre Le chronométrage...  
  
...  
alert("Cliquez pour continuer");  
...  
  
console.timeLog("ma-mesure");        // Affiche Le temps intermédiaire écoulé  
  
...  
alert("Faire d'autres trucs...");  
...  
  
console.timeEnd("ma-mesure");        // Affiche Le temps total écoulé et arrête Le chrono
```

Tableaux

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array

forEach - parcourir les éléments d'un tableau

Parcourir tous les éléments d'un tableau.

```
const arr = ['a', 'b', 'c'];  
  
// Manière fonctionnelle - impossible d'utiliser return ou break !  
arr.forEach((val, index, arr) => { console.log(val, index); });           // a 0, b 1, c 2
```

entries() - parcourir les couples index/valeurs d'un tableau

Parcourir tous les éléments d'un tableau.

```
const arr = ['a', 'b', 'c'];

// style for loop
for (const [index, val] of arr.entries()) { console.log(val, index); } // a 0, b 1, c 2
```

in - parcourir les clés d'un tableau

Parcourir toutes les clés d'un tableau.

```
const arr = ['a', 'b', 'c'];

// boucle for in
for (const key in arr) { console.log(key); } // 0, 1, 2
```

of - parcourir les valeurs d'un tableau

Parcourir toutes les valeurs d'un tableau.

```
const arr = ['a', 'b', 'c'];

// boucle for of
for (const value of arr) { console.log(value); } // a, b, c
```

find() - premier élément qui satisfait une condition

Retourne le premier élément d'un tableau satisfaisant une condition ou **undefined** sinon.

```
const foundElement = elements.find((element) => element.name === 'b');
```

findIndex() - premier index qui satisfait une condition

Retourne l'index du premier élément satisfaisant une condition ou **-1** sinon.

```
const foundElementIndex = elements.findIndex((element) => element.name === 'b');
```

indexOf() et lastIndexOf() - premier/dernier élément qui correspond

Retourne l'indice du premier/dernier élément correspondant à une valeur ou **-1** sinon.

```
const foundElementIndex = elements.indexOf('James');
const foundElementIndex = elements.indexOf('James', 3); // Commence la recherche au 4e élément
```

push(), pop(), shift() et unshift() - ajouter/supprime au début/fin dans un tableau

- **push()** ajoute un élément à la fin du tableau.
- **pop()** supprime le dernier élément du tableau.
- **unshift()** ajoute un élément au début du tableau.
- **shift()** supprime le premier élément du tableau.

```
const array1 = [1, 2, 3];
array1.unshift(4, 5);
console.log(array1); // sortie attendue : Array [4, 5, 1, 2, 3]
```

slice() - ne conserver que certaines lignes d'un tableau

Renvoie une copie superficielle (shallow copy) d'une portion du tableau d'origine, définie par un indice de début (compris) et un indice de fin (exclus).

N.B. Si fin est un indice négatif, il indique un décalage depuis la fin de la séquence.

```
const animals = ["ant", "bison", "camel", "duck", "elephant"];

console.log(animals.slice(2));
// Expected output: Array ["camel", "duck", "elephant"]

console.log(animals.slice(2, 4));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice(1, -1));
// Expected output: Array ["bison", "camel", "duck"]
```

splice() - supprimer/insérer/remplacer des valeurs dans un tableau

Supprime ou ajoute des éléments dans un tableau.

```
var months = ['Jan', 'March', 'April', 'June'];

months.splice(1, 0, 'Feb'); // insertion à l'indice 1
console.log(months); // ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, 'May1', 'May2'); // remplace 1 élément à l'indice 4
console.log(months); // ["Jan", "Feb", "March", "April", "May1", "May2"]
```

concat() - joindre deux tableaux

Joint deux tableaux sans modifier les originaux.

```
const data1 = [1, 2, 3];
const data2 = [10, 20, 30, 40];
const data3 = data1.concat(data2); // data3 = [1, 2, 3, 10, 20, 30, 40]
```

join() - joindre des chaînes de caractères

Retourne une chaîne en concaténant tous les éléments du tableau avec un séparateur.

```
const data = [1, 2, 3];
const values = data.join(', '); // '1, 2, 3'
```

keys() et values() - les clés/valeurs d'un objet

Retourne toutes les clés ou toutes les valeurs d'un objet.

```
const object1 = {
  a: "somestring",
  b: 42,
  c: false,
};

console.log(Object.keys(object1)); // ["a", "b", "c"]
console.log(Object.values(object1)); // ["somestring", 42, false]
```

includes() - vérifier si une valeur est présente dans un tableau

Retourne **true** si la valeur existe dans le tableau.

```
const data = [1, 2, 3];
const isValueInData = data.includes(2);
```

every() et some() - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau

- **every()** retourne **true** si chaque élément passe le test.
- **some()** retourne **true** si au moins un élément passe le test.

```
const data = [1, 2, 3];
const isAllValuesDigits = data.every(elem => (elem < 10) && (elem >= 0)); // true
```

fill() - remplir un tableau avec des valeurs

Remplit un tableau avec des valeurs.

```
const data = [1, 2, 3, 4];

// fill with 0 from position 2 until position 4
const newData = data.fill(0, 2, 4); // [1, 2, 0, 0]

// fill with 0 from position 2 until position 4
const newData = data.fill(6); // [6, 6, 6, 6]
```

flat() - aplatir un tableau

Aplatit un tableau.

```
const twoLevelsDeep = [[1, [2, 2], 1]];

twoLevelsDeep.flat();      // [1, [2, 2], 1]
twoLevelsDeep.flat(2);     // [1, 2, 2, 1]

const veryDeep = [[1, [2, 2, [3,[4,[5,[6]]]]], 1]];
veryDeep.flat(Infinity);   // [1, 2, 2, 3, 4, 5, 6, 1]
```

sort() - pour trier un tableau

Trie un tableau.

```
let data = [1, 2, 3, 4];
data.sort();

let items = [
  { name: "Edward", value: 21 },
  { name: "Sharpe", value: 37 },
  { name: "And", value: 45 },
  { name: "The", value: -12 },
  { name: "Magnetic", value: 13 },
  { name: "Zeros", value: 37 }
];
items.sort(function (a, b) {
  return a.value - b.value;
});
```

map() - tableau avec les résultats d'une fonction

Crée un tableau avec les résultats d'une fonction.

```
const newDataArray = dataArray.map((item) => performSomething(item))

const numbers = [2, 4, 8, 10];
const halves = numbers.map(x => x / 2); // [1, 2, 4, 5]
```

filter() - tableau avec les éléments passant un test

Crée un tableau avec les éléments passant un test.

```
const newDataArray = dataArray.filter((item) => performTest(item))

const words = ["exuberant", "spray", "elite", "destruction", "limit", "present"];
const longWords = words.filter(word => word.length > 6); // LongWords is ["exuberant",
"destruction", "present"]
```

groupBy() - regroupe les éléments d'un tableau selon un règle



Warning

ATTENTION ⚠ : Opérateur **génial** mais disponible depuis peu (>= mars 2024 !!)

Regroupe et ventile les éléments d'un tableau selon une méthode fournie.

```
const inventaire = [
  { produit:"asperges",    categorie:"legume",    nbre:9 },
  { produit:"bananes",     categorie:"fruit",     nbre:5 },
  { produit:"poulpe",      categorie:"poisson",   nbre:3 },
  { produit:"jambon cru",  categorie:"viande",   nbre:9 },
  { produit:"fraises",     categorie:"fruit",     nbre:15 },
  { produit:"cotelette",   categorie:"viande",   nbre:14 },
  { produit:"cerises",     categorie:"fruit",     nbre:22 },
  { produit:"truite",      categorie:"poisson",   nbre:12 },
  { produit:"filet angus", categorie:"viande",   nbre:11 },
  { produit:"carotte",     categorie:"legume",    nbre:62 }
];

const inventaireParGroupes = Object.groupBy(inventaire, ({ nbre }) =>
  nbre < 0 ? "racheter" : "suffisant",
);

console.log(JSON.stringify(inventaireParGroupes));
```

La sortie console sera :

```
{
  "racheter" : [
    { "produit":"asperges",    "categorie":"legume",    "nbre":9 },
    { "produit":"bananes",     "categorie":"fruit",     "nbre":5 },
    { "produit":"poulpe",      "categorie":"poisson",   "nbre":3 },
    { "produit":"jambon cru",  "categorie":"viande",   "nbre":9 }
  ],
  "suffisant" : [
    { "produit":"fraises",     "categorie":"fruit",     "nbre":15 },
    { "produit":"cotelette",   "categorie":"viande",   "nbre":14 },
    { "produit":"cerises",     "categorie":"fruit",     "nbre":22 },
    { "produit":"truite",      "categorie":"poisson",   "nbre":12 },
    { "produit":"filet angus", "categorie":"viande",   "nbre":11 },
    { "produit":"carotte",     "categorie":"legume",    "nbre":62 }
  ]
}
```

flatMap() - chaînage de map() et flat()

Correspond à l'enchaînement de `map()` suivi de `flat()` avec une profondeur 1.

Cela permet donc d'appliquer une fonction à chaque élément du tableau puis d'aplatir le résultat en un tableau.

```
let tableau1 = ["Coucou comment", "", "ça va ?"];

tableau1.map((x) => x.split(" "));
// [["Coucou", "comment"], ["", ""], ["ça", "va", "?"]]

tableau1.flatMap((x) => x.split(" "));
// ["Coucou", "comment", "", "ça", "va", "?"]
```

reduce() et reduceRight() - réduire un tableau à une seule valeur

Réduit un tableau à une seule 'valeur' en appliquant une fonction qui traite chaque valeur d'une liste (de la gauche vers la droite) afin de la réduire à une seule valeur. Une valeur initiale peut être fournie.

Cette fonction géniale va agir comme un « accumulateur » car elle sera à chaque fois appelée en recevant le résultat de son appel précédent avec l'élément du tableau précédent, ce qui lui permettra, si souhaité, d'utiliser ce résultat précédent pour en faire qqch (voir 2ème exemple).

```
// Le total de la liste fait 111, la liste contient 2 multiples de 4
const nbres = [+3, -3, +4, -4, +6, -6, +1, -1, +5, -5, +9, -9, 111 ];

// -----
// 1er exemple : réaliser une somme progressive
// -----
// Sans valeur initiale fournie (zéro par défaut)
const total = nbres.reduce((accumulateur, valeur) => accumulateur + valeur);
// Avec valeur initiale fournie (zéro par défaut)
const total2 = nbres.reduce((accumulateur, valeur) => accumulateur + valeur, 0);

console.log(total); // Le résultat attendu est 111
console.log(total2); // Le résultat attendu est 111

// -----
// 2ème exemple : compter les multiples de 4 sachant qu'il y en avait déjà 48 dans une autre liste et
// qu'on veut rajouter ceux qu'on trouvera à ce nombre
// -----
const nbreTotalMultiples = nbres.reduce((accumulateur, nbre) => ((nbre % 4) === 0) ? accumulateur+1 :
accumulateur, 48); // notez le passage de la valeur initiale de l'accumulateur
console.log(nbreTotalMultiples); // Le résultat attendu est 50

// -----
// 3ème exemple : indexer progressivement "par canton" afin de calculer "final" sur la base de
// "initial"
// -----
const initial = [
  {ville:"Bulle", canton:"FR"},
  {ville:"Lessoc", canton:"FR"},
  {ville:"Lausanne", canton:"VD"},
  {ville:"Riaz", canton:"FR"},
  {ville:"Olten", canton:"BE"},
  {ville:"Montreux", canton:"VD"},
  {ville:"Zermatt", canton:"VS"},
  {ville:"Bern", canton:"BE"}
];

const final = initial.reduce((accumulateur, ville) => {
  (ville.canton in accumulateur) ? accumulateur[ville.canton].push(ville.ville) :
  accumulateur[ville.canton] = [ville.ville];
  return accumulateur;
}, {}); // notez la valeur initiale => objet vide

// const final = {
//   FR: [ 'Bulle', 'Lessoc', 'Riaz' ],
//   VD: [ 'Lausanne', 'Montreux' ],
//   BE: [ 'Olten', 'Bern' ],
//   VS: [ 'Zermatt' ]
// };
```

reverse() - inverser l'ordre du tableau

Inverse l'ordre du tableau.

```
const data1 = [0, 1, 2, 3];
const data2 = data1.reverse(); // [3, 2, 1, 0]
```

Techniques

` (backticks) - pour des expressions intelligentes

Utilisé avec des contenus chaîne de caractères :

```
const firstName = "Derek"
const verb = "develops"
const frequency = "daily"

// sans puis avec les backticks => beaucoup plus clair/propre/lisible !!!
const name = firstName + " " + verb + " " + frequency
const literalName = `${firstName} ${verb} ${frequency}`
```

Utilisé pour éviter trop de caractères d'échappement :

```
Double quotes: "He said, \"Don't do that!\""
Single quotes: 'He said, "Don\'t do that!'"

// avec les backticks => beaucoup plus clair/propre/lisible !!!
Backtick literal: `He said, "Don't do that!"`
```

Utilisé lors de chaîne de caractères multi-lignes :

```
const myUglyString = "<div>\n<p>Hello world!</p>\n</div>";

const myBeautifulString = `<div>
  <p>Hello world!</p>
</div>`;
```

Cas concret d'utilisation :

```
const axios = require("axios")
const clientID = "YOUR_CLIENT_ID"
const secretID = "YOUR_SECRET_ID"
const params = `?client_id=${clientID}&client_secret=${secretID}`
function getProfile(username) {
  return axios.get(`https://api.github.com/users/${username}${params}`).then(
    ({ data }) => data
    // equivalent to function(user) { return user.data }
  )
}
```

new Set() - pour supprimer les doublons

Pour produire un nouveau tableau sans les doublons éventuellement présents.

```
const dupes = [1, 2, 1, 3, 2, 4, 4, 2, 4, 3, 2, 1];
const uniqueSet = new Set( dupes );    // {1, 2, 3, 4}
const noDups = [ ...uniqueSet ];       // Pour retransformer en un tableau à l'aide de l'opérateur "spread"
```

Opérateurs javascript super-coool 😎

opérateur ? :

L'expression `question?valeur1:valeur2` retournera `valeur1` si `question` vaut `true` sinon elle retournera `valeur2`.

```
const age = 15;
const resultat = ( age >= 18 ) ? 'majeur' : 'mineur';    // 'mineur'
```

opérateur ??

Cet opérateur logique se nomme l'opérateur de "coalescence des nuls".

Renvoie son opérande de droite lorsque son opérande de gauche vaut `null` ou `undefined` et qui renvoie son opérande de gauche sinon.

```
const foo1 = null ?? "default";    // "default"
const foo2 = 0 ?? 42;              // 0
```

⚠ Caution

Contrairement à l'opérateur logique OU (`||`), l'opérande de gauche sera également renvoyé s'il s'agit d'une valeur équivalente à `false` et pas seulement `null` et `undefined`.

⚠ En d'autres termes **ATTENTION !!** lors de l'utilisation de `||` pour fournir une valeur par défaut à une variable, car on peut rencontrer des comportements inattendus lorsqu'on considère certaines valeurs comme correctes et utilisables (par exemple une chaîne vide `''` ou `0`) !!

```
const foo3 = 0 || 42;                // 42 => ATTENTION !
const foo4 = 1 || 42;                // 1
const foo5 = null || 'salut !';      // 'salut !'
const foo6 = '' || 'salut !';        // 'salut !' => ATTENTION !
```

opérateur ??=

Cet opérateur logique se nomme l'opérateur d'affectation de "coalescence des nuls", également connu sous le nom d'opérateur affectation logique nulle.

Évalue l'opérande de droite et l'attribue à gauche **UNIQUEMENT** si l'opérande de gauche est nulle (`null` ou `undefined`).

```
const a = { duration: 50 };
a.duration ??= 10;      // pas fait
a.speed ??= 25;         // fait => { duration: 50, speed: 25 }
```

opérateur de décomposition 'spread' ...

L'opérateur de décomposition spread ... permet de décomposer un itérable (comme un tableau) en en ses éléments distincts. Cela permet de rapidement copier tout ou une partie d'un tableau existant dans un autre tableau ou d'en extraire facilement des parties.

```
// Combiner des valeurs existantes dans un nouveau tableau
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];

// Extraire uniquement ce qui est utile d'un tableau
const numbers = [1, 2, 3, 4, 5, 6];
const [one, two, ...rest] = numbers;

// Mariage d'objets avec mise à jour :-)
const myVehicle = {
  brand: 'Ford',
  model: 'Mustang',
  color: 'red'
}
const updateMyVehicle = {
  type: 'car',
  year: 2021,
  color: 'yellow'
}
const myUpdatedVehicle = {...myVehicle, ...updateMyVehicle}
```

Déstructuration

L'opérateur de décomposition spread ... sert aussi à isoler certains éléments afin de les utiliser ensuite, et de **mettre le reste** d'un coup ailleurs.

```
const valeurs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const [a, b, ...c] = valeurs;
console.log(a);    // 1
console.log(b);    // 2
console.log(c);    // [3, 4, 5, 6, 7, 8, 9, 10]
```

Fonctions

Déclaration de fonction

Standard

```
function doStuff(a,b,c) { return a+b+c; };
```

Sous forme d'expression de fonction

```
const doStuff = function(a,b,c) { return a+b+c; }
```

Sous forme d'expression de fonction anonyme

```
const doStuff = (a,b,c) => { return a+b+c; }
```

Sous forme raccourcie

S'il n'y a qu'un seul argument et que son corps n'a qu'une seule expression, on peut omettre le return et le corps de la fonction :

```
const doStuff = (a) => `Salut ${a} !`;
```

Fonctions immédiatement invoquées (IIFE)

IIFE = Immediately Invoked Function Expressions.

Ces fonctions sont définies et **exécutées immédiatement**. Elles sont souvent utilisées pour créer un **contexte isolé** ou encapsuler du code sans polluer l'espace global.

```
(function(){ ... })()
```

ou

```
((() => { ... })())
```